INSTITUTO SEDES SAPIENTIAE
TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE
CÁTEDRA: PROGRAMACIÓN III

ARTFOLIO

DOCUMENTACIÓN TÉCNICA Y MANUAL DE ARQUITECTURA DE SOFTWARE

Plataforma Web Integral de Exhibición Artística Multidisciplinaria con Modelo Zero-Blob, Extracción de Enlaces oEmbed, Previsualización Adaptativa y Panel de Moderación

Docente Evaluador:	Prof. Javier
Estudiante / Desarrollador:	Germán Bianchi
Materia:	Programación III
Entorno de Desarrollo:	Laragon (PHP 8.2+, MySQL 8.0, Apache/Nginx, Mailpit)
Framework Backend & Frontend:	Laravel 11, Laravel Breeze, Blade, Tailwind CSS, Alpine.js
Fecha de Entrega:	Octubre de 2026 — Ciclo Lectivo 2026

ÍNDICE GENERAL

A continuación se presenta la tabla de contenidos exhaustiva que estructura la presente memoria técnica y académica del proyecto Artfolio:

- **1. Resumen Ejecutivo y Propósito del Sistema** — Visión general, justificación académica en Programación III y arquitectura Zero-Blob.
- **2. Patrones de Diseño de Software y Fundamentación Teórica** — Aplicación de MVC, Service Layer, Form Requests, Facade, Strategy y Active Record según Refactoring Guru.
- **3. Arquitectura del Sistema y Pila Tecnológica** — Entorno Laragon, PHP 8.2+, Laravel 11, MySQL 8.0, Tailwind CSS, Alpine.js y servidor SMTP Mailpit.
- **4. Modelo Relacional y Diccionario de la Base de Datos** — Estructura detallada de users, artworks, collections, artwork_collection, bookmarks y reports.
- **5. Estructura Jerárquica del Código Fuente** — Mapeo completo de app/, resources/, routes/ y configuración de middlewares de autorización.
- **6. Análisis Técnico Exhaustivo por Módulo y Funcionalidad** — Detalle de controladores, métodos, FormRequests, reglas de negocio y flujos de ejecución.
- **6.1 Módulo de Exploración Global y Galería Pública** — Búsqueda multi-campo, filtros combinados y resolución adaptativa de imágenes.
- **6.2 Módulo de Gestión de Obras (Artworks)** — Lista blanca estricta, sanitización, render_type y prevención de inyecciones.
- **6.3 Módulo de Colecciones Temáticas** — Relaciones Eloquent N:M, vinculación dinámica y aislamiento de borrado.
- **6.4 Módulo de Perfil de Artista y Recorte de Avatar** — Portafolio público /artist/{slug}, recorte de avatar drag-to-crop interactivo y comisiones.
- **6.5 Módulo de Marcadores y Guardados Privados (Bookmarks)** — Persistencia AJAX reactiva, protección CSRF y privacidad absoluta.
- **6.6 Módulo de Contacto Directo entre Artistas** — Mailable síncrono ContactArtist, seguridad anti-suplantación y bloqueo de auto-contacto.
- **6.7 Módulo de Denuncias y Moderación de Enlaces Caídos** — Flujo de tickets, LinkDownAlert por email, ocultamiento de obras y notas de auditoría.
- **6.8 Módulo de Papelera y Retención a 30 Días** — SoftDeletes, Prunable, restauración desvinculada de obras y vaciado manual.
- **6.9 Panel de Administración General** — Acceso restringido por rol is_admin, KPIs en tiempo real, suspensión y auditoría de artistas.
- **7. Capa de Servicios y Consumo de APIs Externas (EmbedService)** — Protocolo oEmbed, timeout de 4s, adaptabilidad de resolución y mitigación anti-SSRF.
- **8. Frontend, Experiencia de Usuario y Componentes Blade** — Layouts compartidos, modales reactivos adaptativos con Alpine.js e interfaz en inglés.
- **9. Estrategia de Testing Automatizado y Aseguramiento de la Calidad** — Desglose de las 46 pruebas unitarias y de integración con PHPUnit (100% aprobadas).
- **10. Guía de Instalación, Configuración y Despliegue Local** — Paso a paso para clonar, migrar, seedear credenciales y probar en Laragon.
- **11. Conclusiones y Aprendizaje Adquirido** — Reflexión técnica y balance pedagógico del estudiante.

1. RESUMEN EJECUTIVO Y PROPÓSITO DEL SISTEMA

1.1. Visión General del Proyecto

Artfolio es una aplicación web integral desarrollada sobre el framework PHP Laravel 11, concebida para brindar a artistas multidisciplinarios (ilustradores digitales, animadores, modeladores 3D, músicos, diseñadores y escritores) un espacio centralizado donde consolidar y exhibir su producción creativa. En las redes sociales tradicionales y plataformas de portafolio aisladas, los artistas enfrentan la fragmentación de su identidad profesional: sus ilustraciones se alojan en plataformas como DeviantArt o ArtStation, sus bandas sonoras y pistas de audio en Spotify o SoundCloud, sus animaciones en YouTube o Vimeo, y sus modelos tridimensionales interactivos en Sketchfab. Artfolio resuelve esta dispersión mediante una plataforma unificada que permite organizar y vincular obras provenientes de múltiples canales externos.

1.2. El Paradigma Arquitectónico Zero-Blob

Una de las decisiones arquitectónicas fundamentales que distinguen a Artfolio es la adopción del paradigma denominado 'Zero-Blob' (Cero almacenamiento de objetos binarios masivos en el servidor local). Tradicionalmente, las aplicaciones web que manipulan medios audiovisuales e interactivos se enfrentan a severos cuellos de botella: el almacenamiento de archivos de video en alta definición, renders pesados o geometrías 3D satura el disco rígido del servidor, consume excesivo ancho de banda y expone al sistema a vectores de ataque por carga de archivos maliciosos (unrestricted file upload). En lugar de almacenar localmente estos activos, Artfolio almacena exclusivamente metadatos estructurados (URLs canónicas, títulos, descripciones, miniaturas remotas y atributos semánticos) y delega la entrega del contenido multimedia a servidores especializados mediante reproductores embebidos (iframes optimizados y aislados) y protocolos oEmbed.

> 📌 **BENEFICIO ARQUITECTÓNICO ZERO-BLOB:** El enfoque Zero-Blob reduce a cero el consumo de almacenamiento masivo en el servidor web de Laragon, permitiendo que la base de datos MySQL mantenga un tamaño sumamente compacto y de alta velocidad. Toda obra es una referencia referencial a una fuente externa validada mediante lista blanca.

1.3. Justificación Pedagógica en el Marco de Programación III

El desarrollo de Artfolio fue diseñado para articular y consolidar los contenidos nucleares abordados en la asignatura Programación III del Instituto Sedes Sapientiae, a cargo del Profesor Javier. El proyecto no se limita a un CRUD básico de base de datos, sino que implementa un flujo de ingeniería robusto que cubre:

- **Arquitectura Modelo-Vista-Controlador (MVC):** Implementación de controladores tipo Resource y desacoplamiento de lógica.

- **Persistencia y Mapeo Objeto-Relacional:** Uso exhaustivo de Eloquent ORM con relaciones 1 a N, relaciones N a M mediante tablas pivote, Soft Deletes (borrado lógico) y el trait Prunable para purga automática.
- **Form Requests y Validación de Entrada:** Separación de las reglas de validación en clases dedicadas ArtworkRequest y ProfileUpdateRequest con mensajes de error personalizados.
- **Integración de APIs y HTTP Client:** Consumo del cliente HTTP de Laravel (Http::) para solicitar metadatos oEmbed ante servicios de terceros con timeouts y mitigación de ataques SSRF.
- **Sistemas de Plantillas y Componentes:** Estructuración de componentes Blade reutilizables, directivas de control (@auth, @forelse, @if), slots y layouts maestros (app.blade.php y admin.blade.php).
- **Servicio de Notificaciones y Correo:** Envío directo de correos electrónicos síncronos mediante clases Mailable especializadas (ContactArtist y LinkDownAlert) integradas con Mailpit para auditoría local.
- **Aseguramiento de Calidad (Testing Automatizado):** 46 pruebas de integración y unitarias que validan la lógica de negocio, políticas de seguridad y escenarios límite (Edge cases).

2. PATRONES DE DISEÑO Y FUNDAMENTACIÓN TEÓRICA

Siguiendo las pautas de la cátedra de Programación III y los principios expuestos en la literatura estándar de ingeniería de software (particularmente la guía de patrones de diseño de Refactoring Guru), la arquitectura de Artfolio incorpora una serie de patrones reconocidos que promueven la mantenibilidad, legibilidad y desacoplamiento del código.

2.1. Patrón Modelo-Vista-Controlador (MVC)

El patrón arquitectónico MVC es la columna vertebral de la aplicación. En Artfolio, la separación de responsabilidades se mantiene rigurosamente:

- **Modelos (app/Models):** Encapsulan las reglas de datos, relaciones de negocio y lógica de consulta (scopes). Los modelos nunca generan código HTML ni manipulan directamente la sesión HTTP.
- **Controladores (app/Http/Controllers):** Orquestan el flujo de datos. Reciben las peticiones validadas, interactúan con los modelos o servicios y seleccionan la vista de respuesta correspondiente. Se evita la acumulación de lógica de negocio extensa en los controladores delegando tareas complejas a FormRequests y Servicios.
- **Vistas (resources/views):** Renderizan la interfaz de usuario apoyándose en el motor Blade de Laravel. Están libres de consultas SQL directas y se limitan a exhibir los datos provistos por el controlador.

2.2. Patrón Service Layer (Capa de Servicios)

Para encapsular la complejidad de interactuar con plataformas externas (Sketchfab, YouTube, Spotify, DeviantArt, etc.), se implementó el servicio app/Services/EmbedService.php. Este patrón extrae la lógica de comunicación externa fuera de los controladores. De este modo, ArtworkController no necesita conocer las especificaciones del protocolo oEmbed, las expresiones regulares de validación de dominios ni la mitigación de IPs privadas; únicamente invoca el método EmbedService::processUrl($url) y recibe un arreglo estandarizado con la información depurada.

2.3. Patrón Form Request

En concordancia con los contenidos de la materia, la validación de formularios no se realiza de forma rudimentaria mediante métodos inline $request->validate([...]) en los controladores. Se implementaron clases dedicadas como app/Http/Requests/ArtworkRequest.php. Este patrón permite separar la autorización ($this->user() !== null) y las reglas de validación (presencia de campos, longitud, formato de URL y existencia en el enum de categorías) antes de que el código del controlador sea ejecutado.

2.4. Patrón Active Record & Eloquent ORM

Los modelos Artwork, Collection, User y Report heredan de Illuminate\Database\Eloquent\Model, implementando el patrón Active Record. Cada instancia de clase representa un registro en la base

de datos relacional, proveyendo métodos declarativos para definir relaciones (belongsTo, hasMany, belongsToMany) y consultas semánticas avanzadas mediante Query Builders.

2.5. Patrón Strategy / Adapter para Proveedores Multimedia

Cada plataforma externa tiene una estructura de URL y un mecanismo de entrega particular (YouTube utiliza IDs de video y endpoint oEmbed; Spotify utiliza URIs de pistas; Sketchfab utiliza modelos con scripts interactivos; Google Docs utiliza visualizadores de documentos). El servicio EmbedService actúa como un Adapter/Strategy que normaliza estas diferentes fuentes en un único contrato homogéneo compuesto por provider, category, render_type, title, description, thumbnail_url y external_url.

> 📌 **JUSTIFICACIÓN TEÓRICA EN REFACTORING GURU:** Según Refactoring Guru (https://refactoring.guru/es/design-patterns/adapter), el patrón Adapter permite que clases con interfaces incompatibles trabajen juntas. En Artfolio, EmbedService adapta las dispares respuestas JSON de YouTube, Spotify y DeviantArt a un formato de base de datos único e inmutable.

3. ARQUITECTURA DEL SISTEMA Y PILA TECNOLÓGICA

La plataforma Artfolio fue diseñada bajo un stack de tecnologías maduro, moderno y estandarizado en la industria, optimizado para ejecutarse en el entorno local de desarrollo provisto por Laragon en sistemas Windows.

3.1. Ficha de Componentes Tecnológicos

Componente	Tecnología / Versión	Función y Justificación
Lenguaje de Programación	PHP 8.2+ / 8.4+	Tipado estricto, constructor property promotion, match expressions y funciones flecha.
Framework Backend	Laravel 11.x	Estructura simplificada de bootstrap (bootstrap/app.php), Eloquent ORM, migraciones y seeders.
Motor de Base de Datos	MySQL 8.0 (Laragon)	Soporte de claves foráneas con eliminación en cascada, índices FullText y transacciones ACID.
Frontend & Renderizado	Blade + Tailwind CSS (Vite)	Compilación rápida de assets, utilidades CSS puras, soporte de modo oscuro y diseño responsive.
Capa de Interactividad	Alpine.js (v3.x)	Reactividad ligera para modales adaptativos, recorte de avatar interactivo y manejo de eventos.
Servicio de Correo Local	Mailpit (puertos 1025/8025)	Captura de correos SMTP salientes con inspección visual inmediata sin riesgo de spam real.
Pruebas Automatizadas	PHPUnit 11	Suite de pruebas unitarias y de integración sobre base de datos en memoria / refresh database.

3.2. Ciclo de Vida de una Petición HTTP en Artfolio

Cuando un usuario o visitante realiza una interacción en Artfolio, la petición atraviesa las siguientes etapas estandarizadas:

- **1. Punto de Entrada (public/index.php):** El servidor web enruta la solicitud al archivo index.php, el cual inicializa el Service Container de Laravel.
- **2. Pipeline de Middlewares:** Se procesan los middlewares globales (verificación CSRF, sesiones, cookies cifradas). Si la ruta está protegida por auth o can:admin, se valida la identidad del usuario.

- **3. Validación en FormRequest:** Las peticiones que alteran estado (POST, PUT, DELETE) pasan por FormRequests dedicados para validar tipos, longitudes y reglas de negocio.
- **4. Lógica de Controlador y Modelos:** El controlador recupera o persiste datos a través de los modelos Eloquent, delegando a EmbedService si se involucra una URL remota.
- **5. Respuesta (HTML o JSON):** Para solicitudes de navegación, el controlador compila la plantilla Blade correspondiente. Para acciones de interacción asíncrona (marcar favorito o denunciar), retorna una respuesta estructurada en JSON.

4. MODELO RELACIONAL Y DICCIONARIO DE LA BASE DE DATOS

El esquema relacional de Artfolio fue diseñado siguiendo la Tercera Forma Normal (3NF), garantizando la integridad referencial mediante restricciones de clave foránea (Foreign Keys) y facilitando consultas eficientes mediante índices específicos.

4.1. Diccionario de Datos por Tabla

4.1.1. Tabla: users

Almacena la identidad de los usuarios registrados (artistas) y administradores del sistema:

Columna	Tipo de Dato	Restricción	Descripción
id	BIGINT UNSIGNED	PK, Auto-inc	Identificador unívoco del usuario.
name	VARCHAR(255)	NOT NULL	Nombre público o alias del artista.
email	VARCHAR(255)	UNIQUE, NOT NULL	Dirección de correo para login y contacto.
password	VARCHAR(255)	NOT NULL	Hash Bcrypt de la contraseña.
slug	VARCHAR(255)	UNIQUE, NOT NULL	Identificador amigable para URL pública (/artist/{slug}).
bio	TEXT	NULLABLE	Biografía y trayectoria profesional del artista.
commission_status	ENUM('open','closed')	DEFAULT 'closed'	Disponibilidad del artista para encargos comerciales.
avatar_url	VARCHAR(2048)	NULLABLE	URL externa de la fotografía de perfil.
crop_x, crop_y	DECIMAL(5,2)	DEFAULT 0.00	Desplazamiento horizontal y vertical en % para recorte.
crop_zoom	DECIMAL(3,2)	DEFAULT 1.00	Factor de aumento (zoom) del avatar (1.0x - 3.0x).
is_admin	BOOLEAN	DEFAULT FALSE	Bandera de privilegios de administración del sistema.
status	ENUM('active','suspended')	DEFAULT 'active'	Estado de la cuenta para moderación administrativa.

deleted_at	TIMESTAMP	NULLABLE	Marca temporal para borrado lógico (SoftDeletes).

4.1.2. Tabla: artworks

Almacena las obras artísticas vinculadas por los artistas:

Columna	Tipo de Dato	Restricción	Descripción
id	BIGINT UNSIGNED	PK, Auto-inc	Identificador único de la obra.
user_id	BIGINT UNSIGNED	FK -> users(id) ON DELETE CASCADE	Artista creador y propietario de la obra.
title	VARCHAR(255)	NOT NULL	Título de la obra.
description	TEXT	NULLABLE	Descripción de la técnica, concepto o herramientas.
price_hint	VARCHAR(255)	NULLABLE	Precio o valor orientativo (no vinculante).
external_url	VARCHAR(2048)	NOT NULL	URL remota original validada bajo lista blanca.
provider	VARCHAR(50)	NOT NULL	youtube, sketchfab, spotify, deviantart, etc.
category	ENUM('image','video','audio','3d','doc')	NOT NULL	Clasificación de disciplina artística.
render_type	ENUM('image','iframe')	NOT NULL	Modo de presentación visual en el frontend.
thumbnail_url	VARCHAR(2048)	NULLABLE	Miniatura en resolución reducida para carga ágil.
embed_html	TEXT	NULLABLE	Fragmento HTML oEmbed oficial provisto por la API externa.
is_featured	BOOLEAN	DEFAULT FALSE	Marca para destacar la obra en el perfil público.
status	ENUM('published','hidden')	DEFAULT 'published'	Visibilidad de la obra en catálogo y búsquedas.
deleted_at	TIMESTAMP	NULLABLE	Marca temporal de borrado lógico (SoftDeletes de 30 días).

4.1.3. Tablas: collections y artwork_collection (Pivote N a M)

Permiten que un artista organice sus obras en carpetas o series temáticas personalizadas:

Tabla / Columna	Tipo de Dato	Restricción	Propósito
collections.id	BIGINT UNSIGNED	PK, Auto-inc	Identificador de la colección.
collections.user_id	BIGINT UNSIGNED	FK -> users(id) CASCADE	Artista propietario de la colección.
collections.name	VARCHAR(255)	NOT NULL	Nombre descriptivo de la serie temática.
collections.deleted_at	TIMESTAMP	NULLABLE	Borrado suave con retención de 30 días.
artwork_collection.artwork_id	BIGINT UNSIGNED	FK -> artworks(id) CASCADE	Relación pivote a la obra.
artwork_collection.collection_id	BIGINT UNSIGNED	FK -> collections(id) CASCADE	Relación pivote a la colección.

4.1.4. Tablas: bookmarks (Favoritos Privados) y reports (Tickets de Moderación)

Tabla / Columna	Tipo de Dato	Restricción	Propósito
bookmarks.user_id	BIGINT UNSIGNED	FK -> users(id) CASCADE	Usuario que guarda la obra como favorito.
bookmarks.artwork_id	BIGINT UNSIGNED	FK -> artworks(id) CASCADE	Obra marcada en favoritos privados.
reports.id	BIGINT UNSIGNED	PK, Auto-inc	Identificador único del ticket de denuncia.
reports.artwork_id	BIGINT UNSIGNED	FK -> artworks(id) CASCADE	Obra denunciada por enlace caído o contenido.
reports.user_id	BIGINT UNSIGNED	FK -> users(id) NULL ON DELETE	Usuario denunciante (o null si fue eliminado).
reports.reason	VARCHAR(255)	NOT NULL	Motivo declarado ('Broken link', 'Inappropriate', etc.).
reports.admin_note	TEXT	NULLABLE	Notas internas de auditoría registradas por el administrador.

reports.status	ENUM('pending','resolved','dismissed')	DEFAULT 'pending'	Estado del flujo de resolución del reporte.

5. ESTRUCTURA DEL PROYECTO Y ORGANIZACIÓN DE DIRECTORIOS

El código fuente de Artfolio respeta estrictamente la convención estándar de Laravel 11, evitando la sobre-ingeniería innecesaria y facilitando la auditoría del proyecto por parte del cuerpo docente.

```
// Código: Estructura de Directorios del Repositorio
germanBianchi_artfolio/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Admin/
│   │   │   │   ├── DashboardController.php   // KPIs globales y métricas
│   │   │   │   ├── ReportController.php      // Moderación de tickets y alertas
│   │   │   │   ├── TrashController.php       // Papelera global y purga forzada
│   │   │   │   └── UserController.php        // Suspensión y activación de cuentas
│   │   │   ├── ArtworkController.php         // CRUD de obras y previsualizaciones
│   │   │   ├── BookmarkController.php        // Favoritos privados reactivos
│   │   │   ├── CollectionController.php      // Colecciones temáticas N:M
│   │   │   ├── ContactController.php         // Contacto directo y síncrono
│   │   │   ├── ExplorerController.php        // Galería global y buscador multi-campo
│   │   │   ├── ProfileController.php         // Edición de perfil y avatar crop
│   │   │   ├── PublicProfileController.php   // Portafolio público por slug
│   │   │   └── TrashController.php           // Papelera personal del artista
│   │   └── Requests/
│   │       ├── ArtworkRequest.php            // Validación desacoplada de obras
│   │       └── ProfileUpdateRequest.php      // Validación de perfil y avatar
│   ├── Mail/
│   │   ├── ContactArtist.php                 // Correo síncrono para contacto
│   │   └── LinkDownAlert.php                 // Notificación de enlace caído
│   ├── Models/
│   │   ├── Artwork.php                       // Eloquent con SoftDeletes y Prunable
│   │   ├── Collection.php                    // Eloquent con SoftDeletes y Prunable
│   │   ├── Report.php                        // Eloquent para tickets de moderación
│   │   └── User.php                          // Modelo de usuario con autenticación
│   └── Services/
│       └── EmbedService.php                  // Extracción oEmbed y protección SSRF
├── database/
│   ├── migrations/                           // Definición de esquema relacional
│   └── seeders/
│       ├── AdminSeeder.php                   // Aprovisionamiento de superadmin
│       └── DemoContentSeeder.php             // Artistas y obras de prueba
├── resources/
│   └── views/
│       ├── admin/                            // Vistas del panel de administración
│       ├── artworks/                         // Vistas de creación, edición y lista
│       ├── collections/                      // Vistas de colecciones
│       ├── components/                       // Componentes Blade (modales, cards, avatar)
│       ├── emails/                           // Plantillas Blade para correos
│       ├── explorer/                         // Galería pública principal
│       └── layouts/                          // Layouts maestros (app, admin, guest)
├── routes/
│   └── web.php                               // Definición de rutas y middlewares
```

```
└── tests/
    └── Feature/                          // 46 pruebas automatizadas con PHPUnit
```

6. ANÁLISIS TÉCNICO EXHAUSTIVO POR MÓDULO Y FUNCIONALIDAD

A continuación se detalla minuciosamente el funcionamiento interno, lógica de negocio y arquitectura de cada módulo de la aplicación, con sus respectivos controladores, modelos y flujos de ejecución.

6.1. Módulo de Exploración Global y Galería Pública (ExplorerController)

El módulo ExplorerController es el escaparate principal de la plataforma, accesible tanto para visitantes anónimos como para usuarios autenticados. Su objetivo primordial es permitir una navegación fluida a través del catálogo de creaciones publicadas, implementando capacidades de búsqueda en texto libre y filtrado multidimensional.

- **Filtrado de Visibilidad:** Solo se muestran creaciones cuyo status sea 'published' y que no hayan sido enviadas a la papelera (SoftDeleted).
- **Búsqueda Multi-Campo:** El usuario puede ingresar cualquier término. La consulta Eloquent ejecuta coincidencias parciales con LIKE %...% sobre el título de la obra, la descripción de la obra y el nombre del artista autor mediante la relación whereHas('user').
- **Filtros por Categoría:** Permite segmentar por disciplina (image, video, audio, 3d, doc) preservando el término de búsqueda activo.
- **Paginación Preservada:** Se presentan bloques de 12 obras por página con numeración tradicional. Se emplea el método ->withQueryString() para que al avanzar de página no se pierdan los filtros aplicados.
- **Resolución Adaptativa:** Para optimizar el tiempo de carga inicial de la página, la galería presenta miniaturas de baja resolución. Al hacer clic sobre cualquier obra, el visor modal carga el activo en su máxima resolución o el iframe interactivo original.

```php
// Código: PHP - Lógica de Búsqueda y Paginación en ExplorerController
// app/Http/Controllers/ExplorerController.php
public function index(Request $request)
{
    $query = Artwork::where('status', 'published')
        ->with('user');

    if ($request->filled('search')) {
        $term = trim($request->search);
        $query->where(function ($q) use ($term) {
            $q->where('title', 'like', "%{$term}%")
              ->orWhere('description', 'like', "%{$term}%")
              ->orWhereHas('user', function ($uq) use ($term) {
                  $uq->where('name', 'like', "%{$term}%");
              });
        });
    }

    if ($request->filled('category')) {
        $query->where('category', $request->category);
```

```
    }

    $artworks = $query->latest()->paginate(12)->withQueryString();

    return view('explorer.index', compact('artworks'));
}
```

6.2. Módulo de Gestión de Obras (ArtworkController & ArtworkRequest)

Este módulo concentra el ciclo de vida de las obras de arte. Su diseño garantiza que ningún usuario pueda publicar enlaces maliciosos o modificar creaciones ajenas.

- **Validación en Form Request:** Toda petición de creación o actualización es interceptada por ArtworkRequest. Se exige título obligatorio, categoría válida dentro del catálogo y URL externa en formato válido.
- **Validación contra Whitelist:** El controlador delega la URL al servicio EmbedService. Si el dominio no está en la lista blanca oficial (YouTube, Sketchfab, etc.), la petición es abortada con error 422.
- **Asignación Segura del Autor:** La creación se vincula forzosamente a auth()->id(), imposibilitando que un usuario publique a nombre de otro.
- **Políticas de Autorización Estricta:** Los métodos edit, update y destroy verifican que el usuario en sesión sea el propietario legítimo ($artwork->user_id === auth()->id()), emitiendo un código HTTP 403 Forbidden en caso contrario.

```
// Código: PHP - Publicación Segura en ArtworkController
// app/Http/Controllers/ArtworkController.php
public function store(ArtworkRequest $request)
{
    $embedData = $this->embedService->processUrl($request->external_url);

    if (!$embedData) {
        return back()->withInput()->withErrors([
            'external_url' => 'The provided URL is not supported by our verified provider
whitelist.'
        ]);
    }

    $artwork = auth()->user()->artworks()->create([
        'title'         => $request->title,
        'description'   => $request->description,
        'price_hint'    => $request->price_hint,
        'external_url'  => $request->external_url,
        'provider'      => $embedData['provider'],
        'category'      => $request->category,
        'render_type'   => $embedData['render_type'],
        'thumbnail_url' => $embedData['thumbnail_url'],
        'embed_html'    => $embedData['embed_html'],
        'status'        => $request->status ?? 'published',
```

```
    ]);

    return redirect()->route('artworks.index')->with('success', 'Artwork published
successfully!');
}
```

6.3. Módulo de Colecciones Temáticas (CollectionController)

Las colecciones permiten al artista estructurar su portafolio en álbumes temáticos (por ejemplo: 'Arte Conceptual 2026', 'Modelos 3D para Videojuegos', 'Bocetos a Lápiz'). La relación entre obras y colecciones es de muchos a muchos (N:M): una obra puede pertenecer a varias colecciones simultáneamente.

- **Aislamiento de Eliminación:** Si un artista decide eliminar una colección completa, las obras asignadas a dicha serie NO se eliminan ni se envían a la papelera; permanecen activas e intactas en el portafolio general del artista.
- **Sincronización Pivote:** En el formulario de creación/edición de colecciones, el artista selecciona mediante checkboxes qué obras propias desea incluir, sincronizando la tabla pivote mediante el método ->artworks()->sync($request->artworks).

6.4. Módulo de Perfil y Recorte Interactivo de Avatar (ProfileController)

En línea con el modelo Zero-Blob, el perfil de artista permite definir una imagen de avatar mediante una URL externa (por ejemplo, desde Unsplash, GitHub o gravatar). Para ofrecer una experiencia de usuario prémium sin requerir librerías pesadas en el servidor como GD o ImageMagick, se desarrolló un componente interactivo en Alpine.js:

- **1. Enlace Remoto:** El artista ingresa la URL de la imagen externa.
- **2. Arrastre en Frontend:** Un visor circular con overflow: hidden permite al usuario hacer clic y arrastrar la imagen en tiempo real para encuadrar su rostro, ajustando dinámicamente las coordenadas X e Y en porcentaje (-100% a +100%) y el zoom (1.0x a 3.0x).
- **3. Persistencia Ligera:** Al guardar el perfil, los valores crop_x, crop_y y crop_zoom se persisten en la tabla users. Al renderizar el avatar en cualquier vista mediante el componente <x-user-avatar>, se aplican como estilos CSS en línea (transform: translate(...) scale(...)).
- **4. Disponibilidad para Comisiones:** El artista puede indicar si se encuentra disponible para aceptar comisiones o encargos pagos (commission_status = open o closed), mostrándose un badge visual distintivo en su perfil público.

6.5. Módulo de Marcadores y Guardados Privados (BookmarkController)

Artfolio promueve un ambiente creativo sano alejado de las métricas de vanidad. Por ello, la función de 'Favoritos' es estrictamente privada: no existen contadores públicos de 'Me gusta' ni rankings de popularidad que generen presión sobre los artistas.

- **Petición Asíncrona:** El botón de bookmark en la tarjeta de obra envía una solicitud POST asíncrona mediante fetch() con encabezado X-CSRF-TOKEN.
- **Mecanismo Toggle:** Si la obra ya estaba en la lista de favoritos del usuario, se elimina (detach); si no estaba, se agrega (attach). El controlador retorna un JSON con { bookmarked: true/false } y el frontend actualiza el ícono SVG de inmediato.
- **Bandeja de Favoritos:** La vista /bookmarks lista únicamente las creaciones guardadas por el usuario autenticado.

6.6. Módulo de Contacto Directo entre Artistas (ContactController)

Para facilitar oportunidades laborales y colaboraciones profesionales, los artistas pueden enviarse mensajes directos. A diferencia de las arquitecturas que exigen complejas colas en segundo plano, en este proyecto universitario se implementó un despacho de correo directo y síncrono que garantiza confiabilidad absoluta sin requerir demonios externos:

- **Seguridad Anti-Suplantación:** Para enviar un mensaje, el remitente debe haber iniciado sesión. El correo de origen se extrae forzosamente de auth()->user()->email, evitando que un atacante suplante la identidad de otra persona.
- **Bloqueo de Auto-Contacto:** Se valida a nivel de controlador que auth()->id() !== $artist->id, impidiendo que un usuario se auto-envíe mensajes.
- **Despacho Síncrono:** Se ejecuta Mail::to($artist->email)->send(new ContactArtist(...)). En el entorno de desarrollo local, el mensaje es capturado al instante por Mailpit en el puerto 1025.

```php
// Código: PHP - Envío Síncrono de Correo en ContactController
// app/Http/Controllers/ContactController.php
public function send(Request $request, User $artist)
{
    if (auth()->id() === $artist->id) {
        return back()->withErrors(['message' => 'You cannot send a contact message to
yourself.']);
    }

    $validated = $request->validate([
        'subject' => 'required|string|max:150',
        'message' => 'required|string|max:2000',
    ]);

    // Enviar correo síncrono al artista receptor
    Mail::to($artist->email)->send(new ContactArtist(
        sender: auth()->user(),
        artist: $artist,
        subjectText: $validated['subject'],
        messageText: $validated['message']
    ));

    return back()->with('success', 'Your message has been delivered to the artist!');
}
```

6.7. Módulo de Reportes y Moderación de Enlaces Caídos (ReportController & Admin\ReportController)

Al operar bajo el modelo Zero-Blob basado en enlaces externos, existe la posibilidad natural de que un enlace remoto quede inaccesible (video eliminado de YouTube, modelo 3D borrado de Sketchfab, etc.). Artfolio dispone de un sistema de moderación proactivo:

- **1. Denuncia de Usuarios:** Cualquier usuario autenticado puede abrir el modal de reporte y seleccionar el motivo 'Broken link' con detalles opcionales.
- **2. Prevención de Spam:** La base de datos impide que un mismo usuario envíe múltiples reportes pendientes para la misma obra, evitando saturación.
- **3. Bandeja Administrativa:** En /admin/reports, los administradores visualizan la bandeja de tickets con miniaturas, autor y motivo.
- **4. Moderación en un Clic:** Al verificar que el enlace externo está efectivamente caído, el administrador presiona el botón 'Send Broken Link Alert': la obra pasa automáticamente a estado 'hidden' (oculta para el público), el ticket se marca como 'resolved', y el sistema envía un correo al artista autor con el enlace directo para que actualice la URL.

6.8. Módulo de Papelera y Retención a 30 Días (TrashController & Admin\TrashController)

Para evitar pérdidas accidentales de datos, las obras y colecciones implementan el trait SoftDeletes de Eloquent. Cuando un artista elimina una obra, esta no desaparece de la base de datos, sino que se asigna una marca temporal en deleted_at:

- **Papelera de Autoservicio:** En /papelera, el artista puede ver sus obras y colecciones eliminadas con un contador visual de días restantes de gracia.
- **Restauración Desvinculada:** Si la colección a la que pertenecía una obra fue destruida o sigue en papelera, al restaurar la obra el sistema desvincula automáticamente la relación pivote huérfana, dejando la obra activa y visible en el portafolio general sin generar errores de clave foránea.
- **Purga a 30 Días:** Los modelos implementan el trait Prunable de Laravel. Cualquier elemento con más de 30 días en papelera (deleted_at <= now()->subDays(30)) es eliminado de forma física y definitiva al ejecutarse el comando php artisan model:prune.

6.9. Módulo de Administración General (Admin\DashboardController & Admin\UserController)

El panel de administración en /admin provee supervisión global del estado de la plataforma:

- **Tarjetas KPI:** Conteo dinámico en tiempo real de artistas registrados, obras activas, tickets de reporte pendientes y elementos en papelera global.
- **Gestión de Usuarios:** Lista paginada de todos los usuarios registrados con filtros por rol (artista / admin) y estado. Permite suspender temporalmente a un usuario para impedir que publique obras o restaurar su acceso.
- **Seguridad de Acceso:** Todas las rutas dentro del prefijo /admin están blindadas por el middleware can:admin y la verificación del campo is_admin === true en la base de datos.

7. CAPA DE SERVICIOS Y CONSUMO DE APIS EXTERNAS (EmbedService)

El componente app/Services/EmbedService.php es el motor central del procesamiento multimedia en Artfolio. Este servicio implementa tres capas de responsabilidad crítica: validación de lista blanca, mitigación de ataques SSRF y extracción de metadatos oEmbed.

7.1. Lista Blanca Estricta de Dominios Autorizados

Para evitar que la plataforma sea utilizada como vector de phishing, inyección de scripts o carga de contenidos no deseados, se definió una lista blanca estricta de dominios que cubre las principales redes artísticas del mundo:

- **Plataformas de Video:** youtube.com, youtu.be, vimeo.com (Clasificación: video)
- **Modelado e Interactividad 3D:** sketchfab.com (Clasificación: 3d)
- **Música y Audio:** spotify.com, soundcloud.com (Clasificación: audio)
- **Ilustración e Imagen:** deviantart.com, pinterest.com, pin.it, artstation.com (Clasificación: image)
- **Documentos y Literatura:** docs.google.com, drive.google.com (Clasificación: doc)

7.2. Mitigación de Vulnerabilidades Server-Side Request Forgery (SSRF)

Cuando un servidor web realiza peticiones HTTP hacia URLs provistas por usuarios, existe el riesgo de ataques SSRF, donde un atacante ingresa una dirección interna (como http://127.0.0.1:3306 o http://169.254.169.254) para escanear servicios locales del servidor o extraer metadatos de instancias en la nube. EmbedService neutraliza esta amenaza resolviendo el host y rechazando categóricamente:

- **Direcciones Loopback:** 127.0.0.0/8, localhost, ::1
- **Direcciones IP Privadas:** 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- **Direcciones Link-Local:** 169.254.0.0/16 (Link-Local y metadatos de AWS/GCP)
- **Direcciones Especiales:** 0.0.0.0 y esquemas que no sean estrictamente HTTP o HTTPS.

7.3. Consumo del Protocolo oEmbed y Resolución Adaptativa de Imagen

El servicio realiza llamadas mediante Http::timeout(4)->get(...) contra los endpoints oEmbed oficiales de los proveedores autorizados. Adicionalmente, implementa una lógica de resolución adaptativa de imágenes: para plataformas como Pinterest, DeviantArt o YouTube, extrae tanto una versión optimizada y liviana de miniatura para la cuadrícula del catálogo como el enlace de alta resolución para el visor modal desplegable.

8. FRONTEND, EXPERIENCIA DE USUARIO Y COMPONENTES BLADE

La interfaz de Artfolio fue concebida bajo principios modernos de diseño de aplicaciones web, combinando la robustez de Blade con la elegancia visual de Tailwind CSS y la reactividad instantánea de Alpine.js.

8.1. Componente Modal Reactivo Adaptativo (<x-viewer-modal>)

A diferencia de los reproductores estáticos convencionales, el modal de visualización de Artfolio adapta su relación de aspecto y controles según la disciplina de la obra:

- **Obras 3D y Video:** Se presenta un contenedor panorámico con aspecto 16:9 optimizado para la interacción fluida a 60 FPS.
- **Obras de Audio:** Se renderiza una barra compacta con controles de reproducción dedicados de Spotify o SoundCloud.
- **Obras Escritas y Documentos:** Se integra un visor vertical de 80vh con controles de desplazamiento de Google Docs.
- **Ilustraciones e Imágenes:** Se exhibe la imagen escalada en alta resolución acompañada de un botón prominente 'View original link' para visitar la publicación del autor.

8.2. Componente Modal Interceptor para Visitantes (<x-guest-modal>)

En la mayoría de aplicaciones web, cuando un visitante anónimo hace clic en 'Guardar favorito', 'Reportar' o 'Contactar al artista', el sistema ejecuta una redirección HTTP 302 hacia la pantalla de login. Esto destruye la experiencia de navegación del usuario, haciéndole perder la posición en la galería y el contexto de lo que estaba mirando.

Artfolio implementa el componente <x-guest-modal>. Mediante un evento global de JavaScript (window.dispatchEvent(new CustomEvent('open-guest-modal'))), cualquier intento de interacción restringida abre una elegante ventana emergente que le informa:

> 📌 **MENSAJE DEL INTERCEPTOR DE INVITADOS:** "Inicia sesión o crea una cuenta gratis para guardar esta obra en favoritos, contactar al artista o enviar un reporte. Permanecerás exactamente en esta página luego de ingresar."

El visitante puede iniciar sesión o registrarse manteniendo intacta su ubicación dentro del portafolio del artista.

8.3. Interfaz de Usuario en Inglés (English UI)

Para responder a los estándares internacionales de plataformas de portafolio y permitir que el proyecto sea comprendido por evaluadores de cualquier procedencia, la totalidad de los textos de la interfaz pública y privada (menús, botones, badges, mensajes flash, tablas y modales) fueron traducidos e implementados en idioma inglés.

9. ESTRATEGIA DE TESTING AUTOMATIZADO Y ASEGURAMIENTO DE LA CALIDAD

Para garantizar que ninguna modificación o refactorización rompa funcionalidades preexistentes, Artfolio cuenta con una suite completa de 46 pruebas automatizadas construidas sobre PHPUnit y el trait RefreshDatabase de Laravel.

Clase de Prueba	Pruebas	Asercion es	Escenarios Críticos Validados
AdminManualReportTest	3	12	Envío de denuncias, bloqueo de reportes duplicados pendientes, control de acceso 403 a usuarios no admin, ocultamiento de obras al enviar alerta y envío síncrono del correo LinkDownAlert al artista.
AdminSeederTest	2	7	Aprovisionamiento correcto de credenciales del administrador y verificación de idempotencia en ejecuciones sucesivas del seeder.
ArtworkEmbedTest	4	35	Aceptación de dominios autorizados en whitelist, rechazo de dominios no autorizados (HTTP 422), validación en ArtworkRequest y mitigación estricta de ataques SSRF contra IPs privadas y loopback.
ExplorerSearchTest	4	16	Filtrado estricto de obras publicadas (exclusión de borradores y papelera), búsqueda multi-campo (título, descripción, artista), filtros por categoría y paginación en bloques de 12 con withQueryString().
GuestInterceptorTest	4	8	Rechazo con código HTTP 401 a visitantes que intentan guardar favoritos, contactar o reportar por JSON; presencia activa del componente x-guest-modal en vistas públicas.
TrashIndependentTest	4	14	Preservación de registros en base de datos con SoftDeletes, aislamiento de borrado en colecciones, restauración de obras huérfanas desvinculadas de colecciones y purga a 30 días con model:prune.
ProfileTest & Breeze Suite	25	73	Ciclo completo de autenticación: Login, Register, ConfirmPassword, ForgotPassword, ResetPassword, UpdateProfile y eliminación de cuenta protegida por contraseña.

📌 **CERTIFICACIÓN DE PRUEBAS AUTOMATIZADAS:** Resultado de la última ejecución oficial en consola:
Tests: 46 passed (46 total)

> Assertions: 165 passed
> Duration: 15.89s — Tasa de éxito: 100.0%

10. GUÍA DE INSTALACIÓN, CONFIGURACIÓN Y DESPLIEGUE LOCAL

A continuación se presenta el procedimiento paso a paso para desplegar y verificar el funcionamiento de Artfolio en el entorno de evaluación docente:

10.1. Requisitos Previos del Sistema

- • **Intérprete PHP:** PHP 8.2 o superior con extensiones pdo_mysql, mbstring, curl, openssl.
- • **Base de Datos:** Servidor de base de datos MySQL 8.0 activo en Laragon.
- • **Gestor Composer:** Composer 2.x para gestión de dependencias PHP.
- • **Gestor Node/NPM:** Node.js (v18+) y NPM para compilación de estilos Tailwind CSS con Vite.
- • **Servidor SMTP Local:** Mailpit ejecutándose en los puertos 1025 (SMTP) y 8025 (Web UI).

10.2. Procedimiento de Puesta en Marcha

```
// Código: Comandos Bash / PowerShell para Despliegue
# 1. Posicionarse en el directorio raíz del proyecto
cd c:\laragon\www\germanBianchi_artfolio

# 2. Instalar dependencias de PHP y JavaScript (si fuera necesario)
composer install
npm install

# 3. Compilar los assets frontend de producción
npm run build

# 4. Ejecutar la migración limpia de la base de datos y cargar seeders
php artisan migrate:fresh --seed

# 5. Ejecutar la suite completa de pruebas para verificar integridad
php artisan test

# 6. Iniciar el servidor local de desarrollo (si no se utiliza VirtualHost de Laragon)
php artisan serve
```

10.3. Cuentas de Acceso Predeterminadas para Evaluación

El seeder inicial crea automáticamente las siguientes cuentas con contraseña estandarizada 'password':

Rol / Perfil	Nombre de Usuario	Correo Electrónico	Contraseña
Superadministrador	Germán Bianchi (Admin)	admin@artfolio.test	password
Artista Digital 3D	Elena Rostova	elena@artfolio.test	password

Concept Artist	Marcus Vance	marcus@artfolio.test	password
Compositora Musical	Sofía Alarcón	sofia@artfolio.test	password
Ilustrador Tradicional	Lucas Morel	lucas@artfolio.test	password

11. CONCLUSIONES Y APRENDIZAJE ACADÉMICO

El desarrollo del proyecto Artfolio representó una experiencia de aprendizaje técnico sumamente enriquecedora, permitiendo conectar los conceptos teóricos dictados en la materia Programación III con un producto de software real, funcional y seguro.

11.1. Logros Técnicos Destacados

- **Dominio del Modelo Zero-Blob:** Se demostró que es posible construir una plataforma de exhibición de medios de alto impacto sin almacenar archivos pesados en el servidor local, reduciendo los costos de infraestructura a cero y maximizando la velocidad de respuesta.
- **Profundidad en Eloquent ORM:** Se comprendió el ciclo de vida completo de Eloquent, aplicando relaciones complejas, tablas pivote, borrado lógico con SoftDeletes y purga programada mediante el trait Prunable.
- **Seguridad y Protección contra Ataques:** La implementación de listas blancas, sandboxing en iframes y mitigación de ataques SSRF permitió entender las implicancias reales de la seguridad defensiva en aplicaciones web modernas.
- **Cultura de Calidad y Testing:** La creación y mantenimiento de una suite de 46 pruebas automatizadas con 100% de efectividad brindó la confianza necesaria para refactorizar y simplificar el código sin temor a introducir regresiones.

11.2. Balance Final

Artfolio no es únicamente una entrega académica; es una demostración integral de las capacidades del framework Laravel 11 utilizado con criterio, simplicidad y apego a las buenas prácticas de la ingeniería de software. El código resultante es limpio, legible y auténtico a la formación impartida por la cátedra, listo para ser evaluado y defendido ante el cuerpo docente.